
ECSE 4660/6660: Internetworking of Things

Lecture 14: Network Layer – Part II:
IP addressing, DHCP, ARP

Ish Jain
ECSE, RPI

Spring 2026

Administrative

- Use Piazza for any questions
 - <https://piazza.com/rpi/spring2026/ecse46606660>
- Today's Quiz is available on **LMS**
 - Multiple choice questions
 - Due tonight 11:59 PM
- Lab
 - TA hours 4-5 PM daily in 5312 lab
 - Lab 3 due Mon, 3/9 (BLE, Zbee)
 - Lab 4 due Fri, 3/20 (DSR, AODV, ARP)
- HW
 - HW3 release today, due Mon, 3/16
 - No Late submission allowed
- Remote class 2/26, ~~3/23~~ 3/30

- Project group formation
 - [IoT_SP26_project.xlsx](#)
- Project Next Steps
 - Work with TA to get the hardware needed (respond on piazza)
 - Mid-term report due 3/20 (2 weeks)
 - In-class presentation+2min video 4/20, 4/23
 - Live-demo 4/27
 - Final code/report 5/1

Toward a Calculus for Compound AI to Understand Complex Systems

Lav Varshney

Della Pietra Infinity Professor and Director of the AI
Innovation Institute
Stony Brook University

ECSE Seminar Series

JEC 3117

Mon, March 09, 2026 at 4:00 PM

Understanding complex system dynamics may require putting together several AI models and methods into compound AI systems that are complex configurations themselves. While individual AI models may excel across diverse domains—from protein analysis to climate prediction—such composition may enable enhanced performance, efficiency, interpretability, and inclusivity. We present several concrete approaches to demonstrate this paradigm. Equituning efficiently converts pretrained models into group-equivariant forms, improving compositional generalization. Platonic grounding optimizes multimodal AI models by processing multimodal tokens only in deeper layers where natural cross-modal alignment occurs, enhancing efficiency by reducing redundancy. Information lattice learning provides intrinsically interpretable knowledge discovery through hierarchical concept learning from universal priors. Efficient federated finetuning and federated digital twins enable bringing together data sources that may be restricted otherwise. We close by discussing how these exemplary concrete approaches might be thought of in a unified calculus for building larger intelligent systems.

Lav Varshney is the Della Pietra Infinity Professor and inaugural director of the AI Innovation Institute at Stony Brook University. He is co-founder and CEO of Kocree, Inc., a startup company using novel human-integrated AI for creativity and scientific discovery, and chief scientist of Ensaras, Inc., a startup company focused on AI and wastewater treatment. He holds appointments at Brookhaven National Laboratory and at RAND Corporation. He has previously been a faculty member at the University of Illinois Urbana-Champaign, a visiting scholar at the Kellogg School of Management, a research staff member at IBM Research, and a principal research scientist at Salesforce Research AI. He is a former White House staffer, having served on the National Security Council staff as a White House Fellow, where he contributed to national AI and wireless communications policy. His work and public scholarship has been featured in media ranging from Fox News and the Wall Street Journal to the New York Times, NPR, Slate, and The New Yorker. He appeared in the Robert Downey, Jr. documentary series, *Age of AI*. His research interests include information theory and artificial intelligence. He received his B.S. degree from Cornell University and his S.M. and Ph.D. degrees from the Massachusetts Institute of Technology.



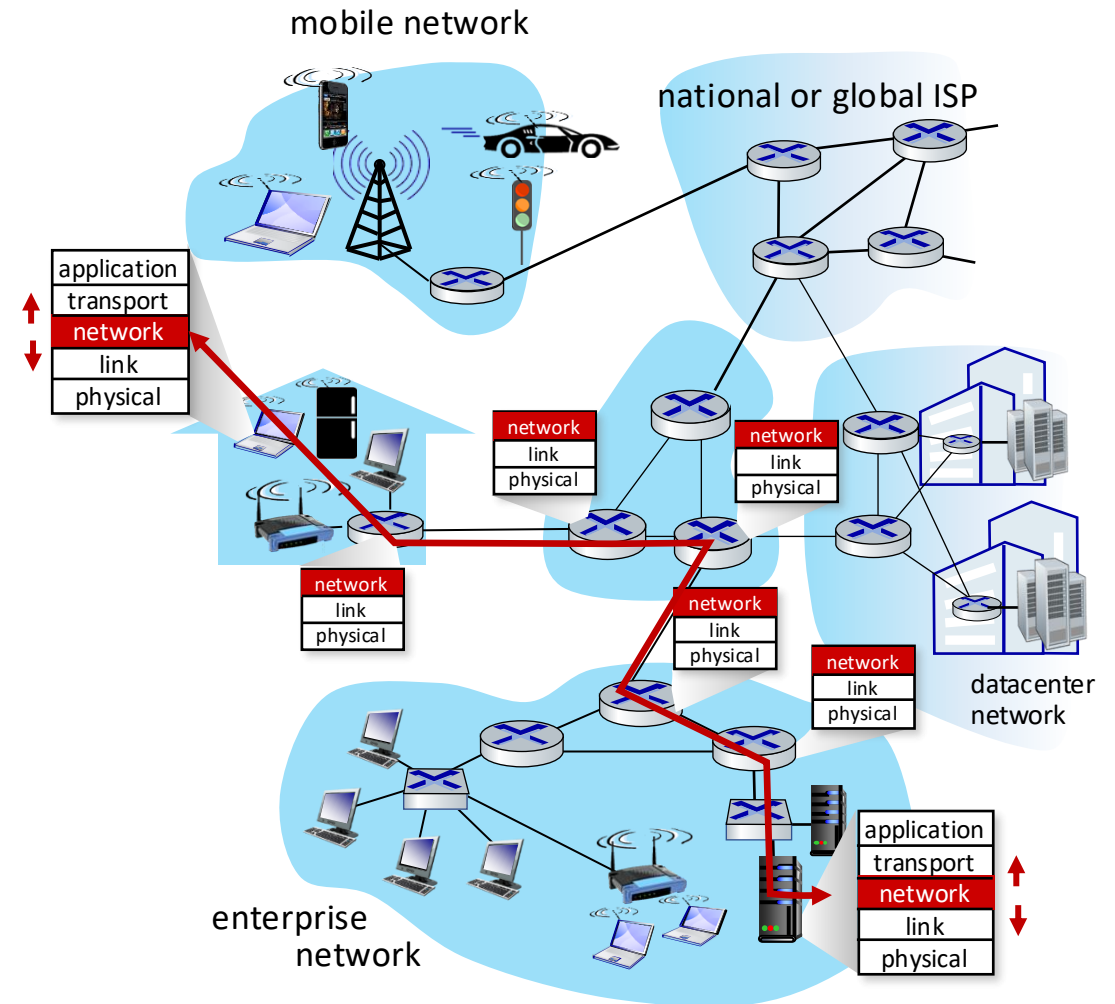
Network layer: “data plane” roadmap

- Prev class: DSR/AODV Routing protocol
- Network layer: overview
- IP: the Internet Protocol
 - Datagram format and IP Addressing
 - DHCP
 - ARP



Network-layer services and protocols

- transport segment from sending to receiving host
 - **sender:** encapsulates segments into datagrams, passes to link layer
 - **receiver:** delivers segments to transport layer protocol
- network layer protocols in *every Internet device*: hosts, routers
- **routers:**
 - examines header fields in all IP datagrams passing through it
 - moves datagrams from input ports to output ports to transfer datagrams along end-end path



Two key network-layer functions

network-layer functions:

- *forwarding*: move packets from a router's input link to appropriate router output link
- *routing*: determine route taken by packets from source to destination
 - *routing algorithms*

analogy: taking a trip

- *forwarding*: process of getting through single interchange
- *routing*: process of planning trip from source to destination



forwarding



routing

Network-layer service model

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no

Internet “best effort” service model

No guarantees on:

- successful datagram delivery to destination
- timing or order of delivery
- bandwidth available to end-end flow

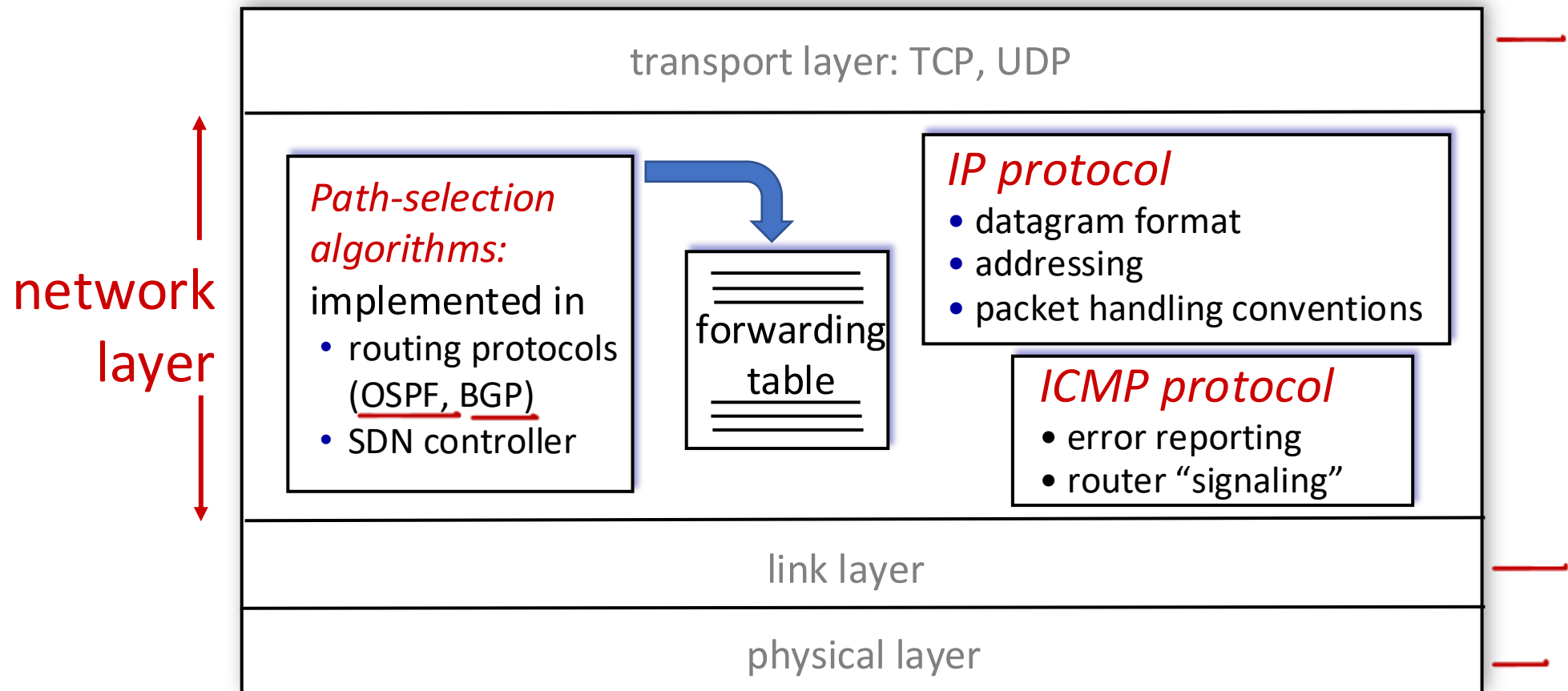
Network layer: “data plane” roadmap

- Network layer: overview
- IP: the Internet Protocol
 - Datagram format and IP Addressing
 - DHCP
 - ARP



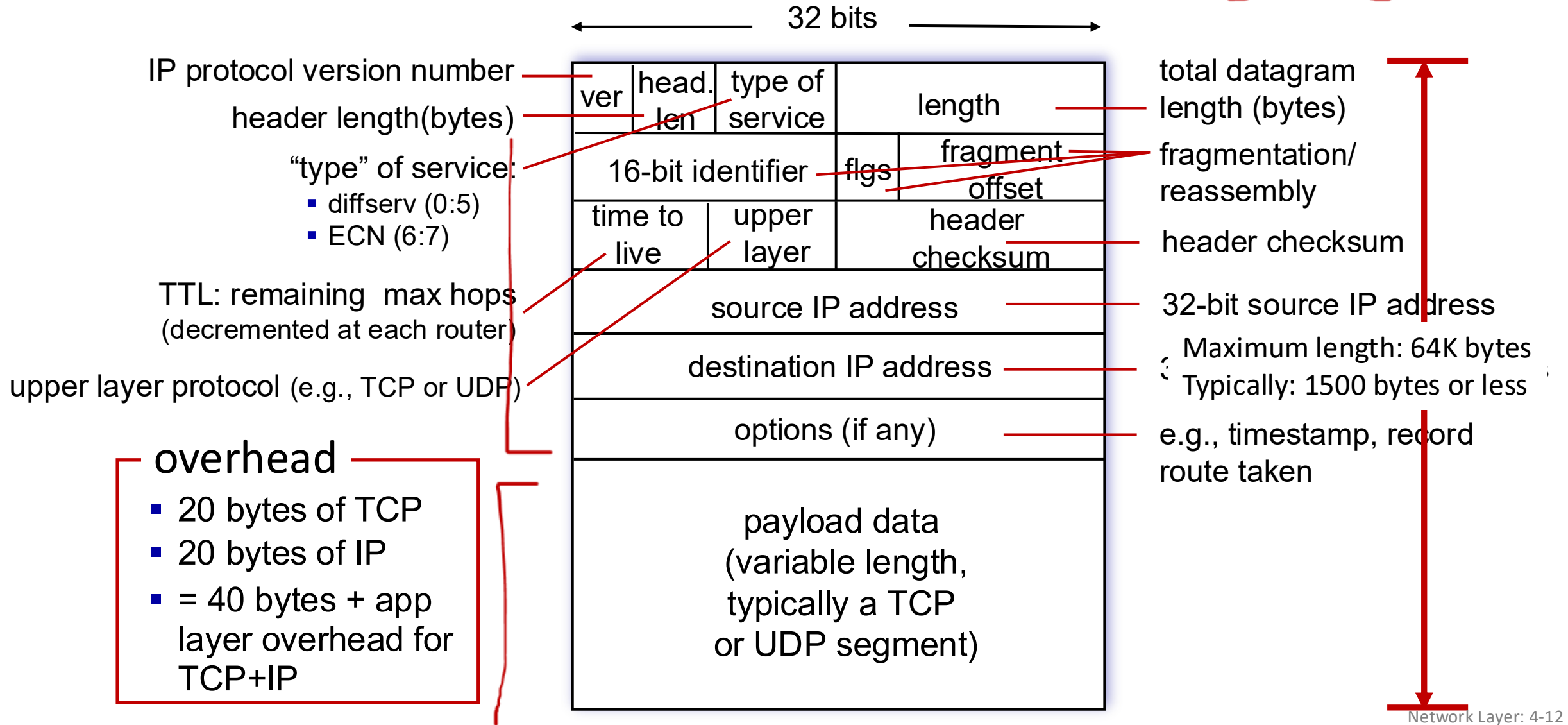
Network Layer: Internet

host, router network layer functions:



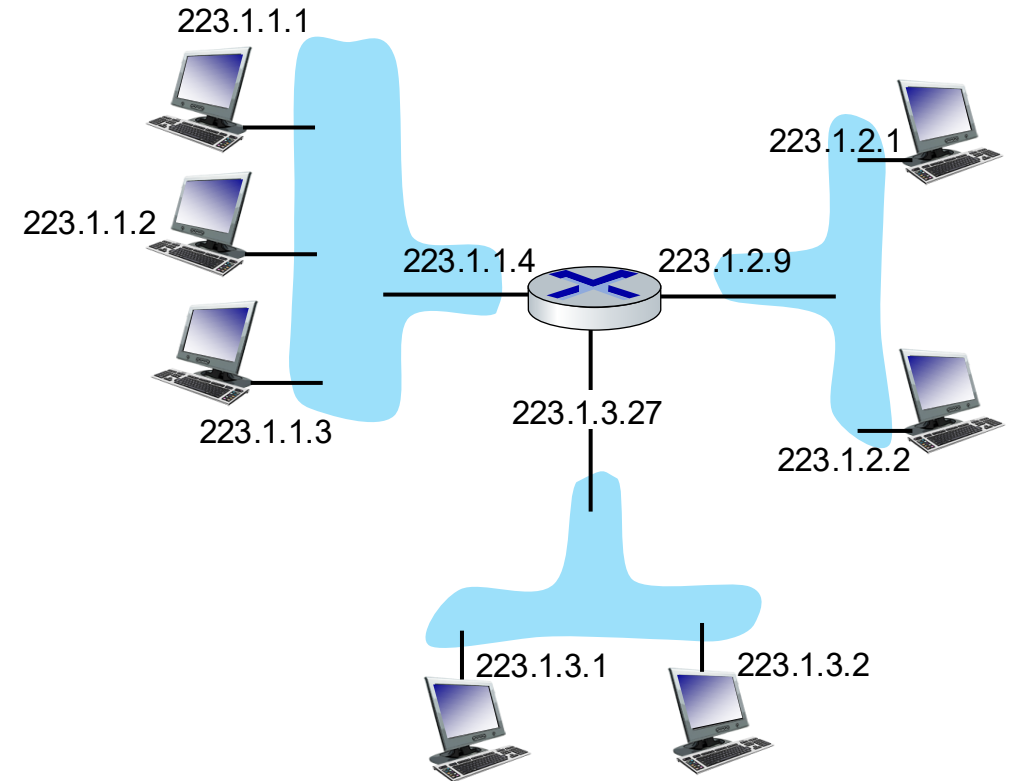
IP Datagram format

$5 \times 4 = 20 \text{ Byte}$



IP addressing: introduction

- **IP address:** 32-bit identifier associated with each host or router *interface*
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



dotted-decimal IP address notation:

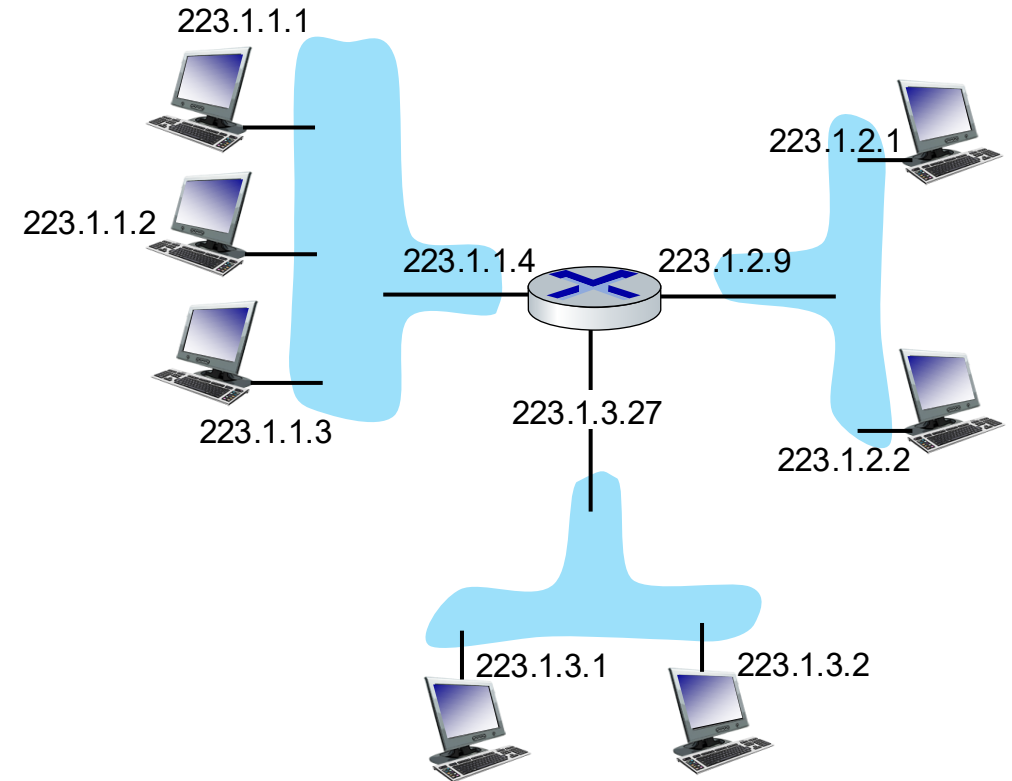
223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

Network Layer: 4-13

IP addressing: introduction

- **IP address:** 32-bit identifier associated with each host or router *interface*
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



dotted-decimal IP address notation:

223.1.1.1 = 11011111 00000001 00000001 00000001

223

1

1

1

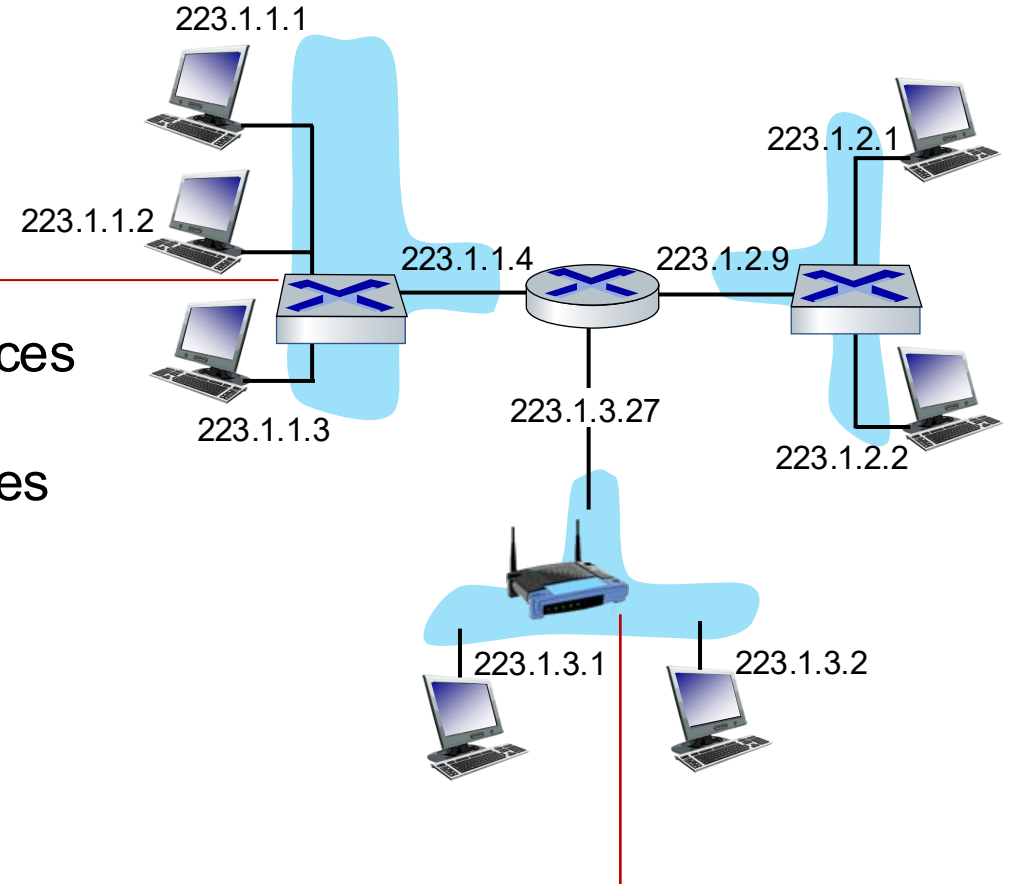
Network Layer: 4-14

IP addressing: introduction

Q: how are interfaces
actually connected?

A: Link Layer

A: wired
Ethernet interfaces
connected by
Ethernet switches



A: wireless WiFi interfaces
connected by WiFi base station

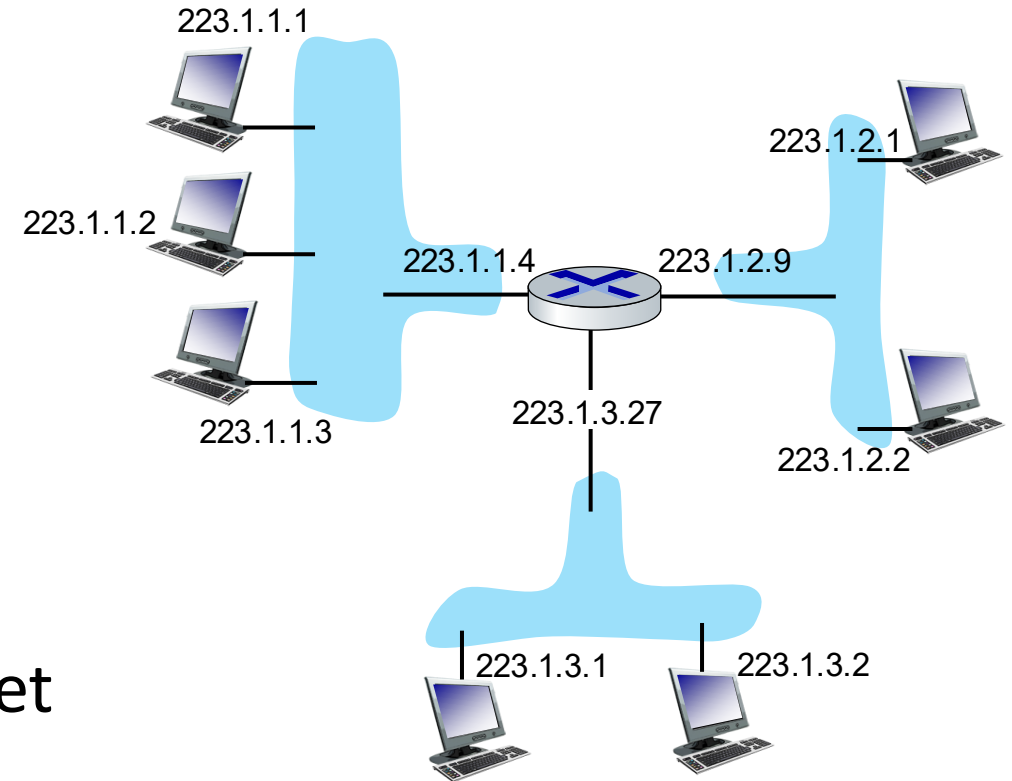
Subnets

■ *What's a subnet ?*

- device interfaces that can physically reach each other **without passing through an intervening router**

■ IP addresses have structure:

- **subnet part:** devices in same subnet have common high order bits
- **host part: remaining** low order bits

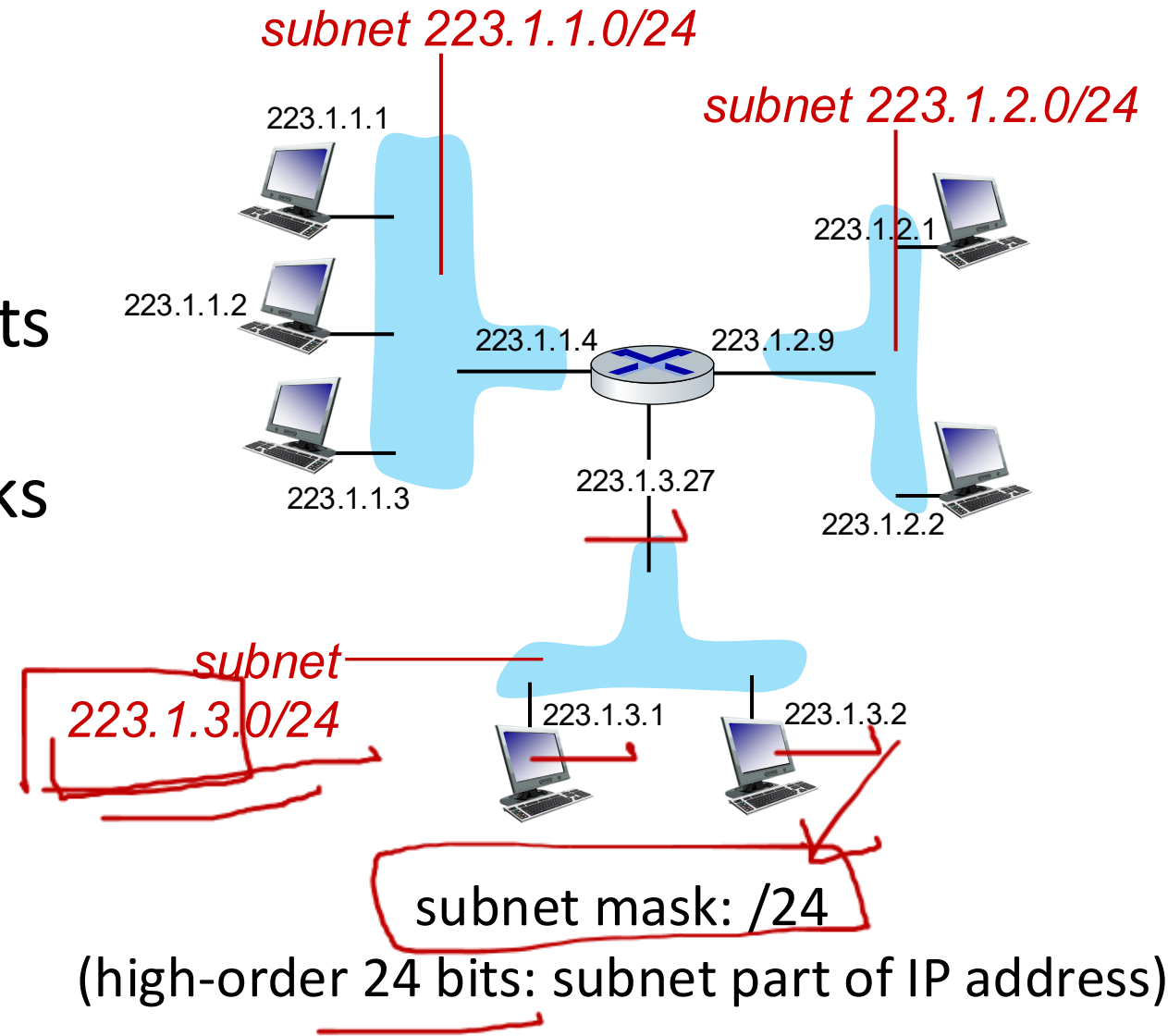


network consisting of 3 subnets

Subnets

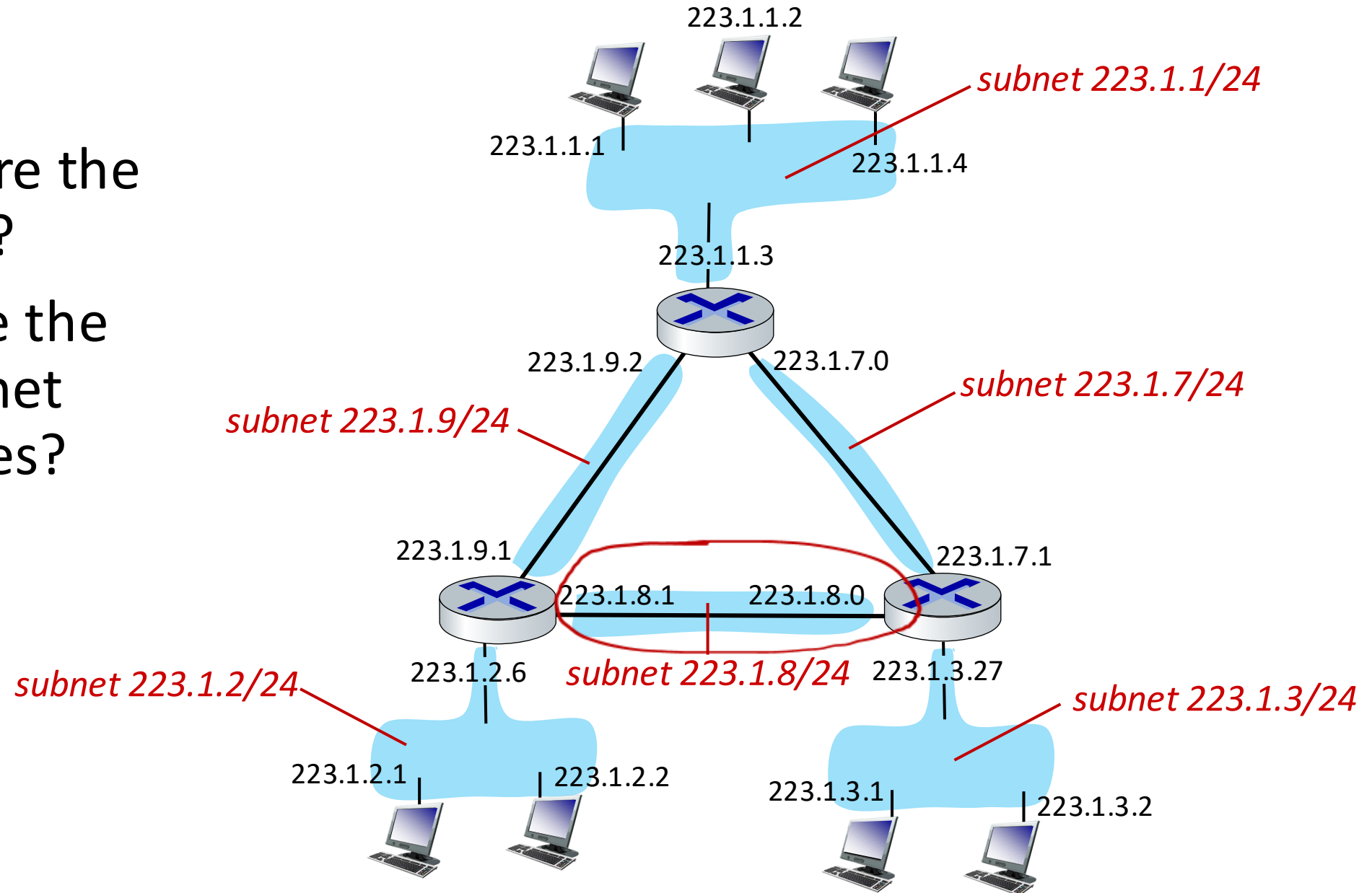
Recipe for defining subnets:

- detach each interface from its host or router, creating “islands” of isolated networks
- each isolated network is called a *subnet*



Subnets

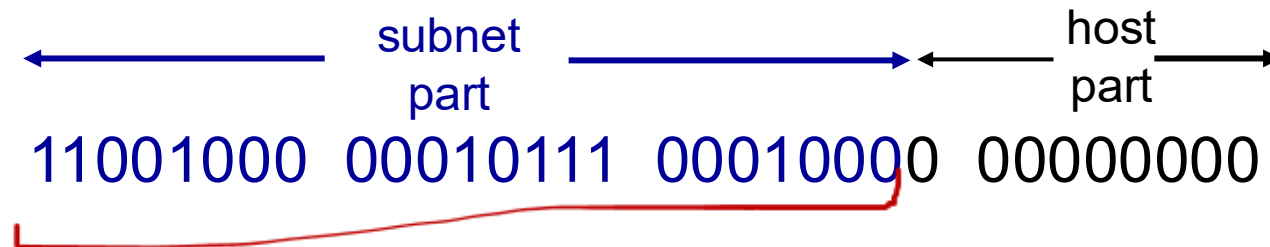
- where are the subnets?
- what are the /24 subnet addresses?



IP addressing: CIDR

CIDR: Classless InterDomain Routing (pronounced “cider”)

- subnet portion of address of arbitrary length
- address format: a.b.c.d/x, where x is # bits in subnet portion of address



200.23.16.0/23

/ 23

IP addresses: how to get one?

That's actually **two** questions:

1. Q: How does a *host* get IP address within its network (host part of address)?
2. Q: How does a *network* get IP address for itself (network part of address)

How does *host* get IP address?

- hard-coded by sysadmin in config file (e.g., /etc/rc.config in UNIX)
- **DHCP**: **D**ynamic **H**ost **C**onfiguration **P**rotocol: dynamically get address from as server
 - “plug-and-play”

Destination-based forwarding

forwarding table

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010000 00000100 through 11001000 00010111 00010000 00000111 11001000 00010111 00011000 11111111	n 3
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Q: but what happens if ranges don't divide up so nicely?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use longest address prefix that matches destination address.

Destination Address Range				Link interface
11001000	00010111	00010***	*****	0
11001000	00010111	00011000	*****	1
11001000	00010111	00011***	*****	2
otherwise				3

examples:

11001000 00010111 00010110 10100001 which interface?

11001000 00010111 00011000 10101010 which interface?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 match! 1 00011*** *****	2
otherwise	3

examples:

11001000 00010111 00010110 10100001 which interface?
11001000 00010111 00011000 10101010 which interface?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range				Link interface
11001000	00010111	00010***	*****	0
11001000	00010111	00011000	*****	1
11001000	00010111	00011***	*****	2
otherwise				3

match!

examples:

11001000	00010111	00010110	10100001	which interface?
11001000	00010111	00011000	10101010	which interface?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

match!

examples:

11001000 00010111 00010110 10100001	which interface?
11001000 00010111 00011000 10101010	which interface?

Longest prefix matching

- we'll see *why* longest prefix matching is used shortly, when we study addressing
- longest prefix matching: often performed using ternary content addressable memories (TCAMs)
 - *content addressable*: present address to TCAM: retrieve address in one clock cycle, regardless of table size
 - Cisco Catalyst: ~1M routing table entries in TCAM

Network layer: “data plane” roadmap

- Network layer: overview
- IP: the Internet Protocol
 - Datagram format and IP Addressing
 - DHCP
 - ARP

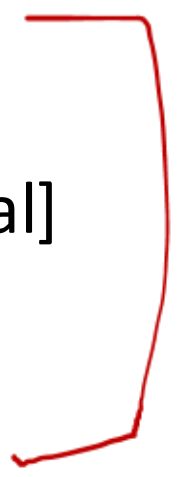


DHCP: Dynamic Host Configuration Protocol

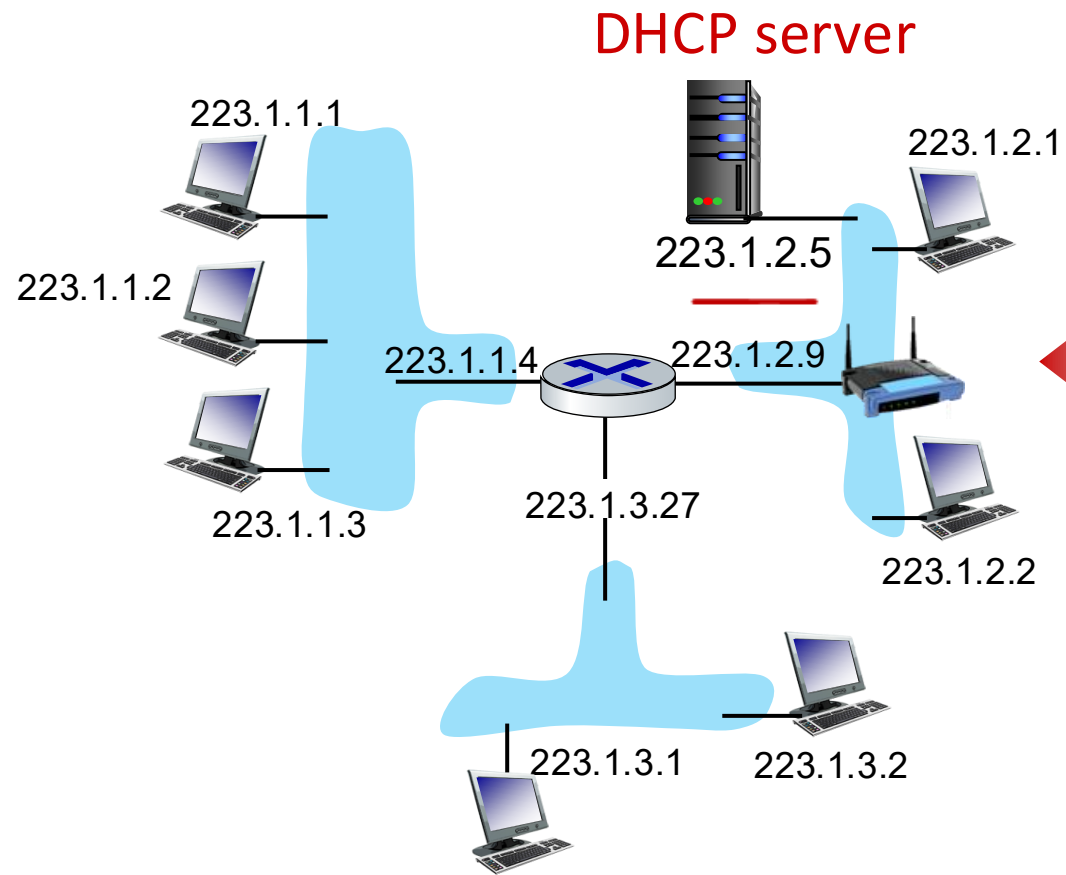
goal: host *dynamically* obtains IP address from network server when it “joins” network

- can ~~renew its lease~~ on address in use
- allows reuse of addresses (only hold address while connected/on)
- support for mobile users who join/leave network

DHCP overview:

- host broadcasts **DHCP discover** msg [optional]
 - DHCP server responds with **DHCP offer** msg [optional]
 - host requests IP address: **DHCP request** msg
 - DHCP server sends address: **DHCP ack** msg
- 

DHCP client-server scenario



Typically, DHCP server will be co-located in router, serving all subnets to which router is attached

arriving **DHCP client** needs address in this network

DHCP client-server scenario

DHCP server: 223.1.2.5



DHCP discover

✓ Broadcast: is there a DHCP server out there?

Arriving client



DHCP offer

Broadcast: I'm a DHCP server! Here's an IP address you can use

DHCP request

Broadcast: OK. I would like to use this IP address!

DHCP ACK

Broadcast: OK. You've got that IP address!

The two steps above can be skipped "if a client remembers and wishes to reuse a previously allocated network address" [RFC 2131]

DHCP client-server scenario

DHCP server: 223.1.2.5



DHCP discover

src : 0.0.0.0, 68
dest.: 255.255.255.255, 67
yiaddr: 0.0.0.0
transaction ID: 654

Arriving client



DHCP offer

src: 223.1.2.5, 67
dest: 255.255.255.255, 68
yiaddr: 223.1.2.4
transaction ID: 654
lifetime: 3600 secs

DHCP request

src: 0.0.0.0, 68
dest.: 255.255.255.255, 67
yiaddr: 223.1.2.4
transaction ID: 655
lifetime: 3600 secs

DHCP ACK

src: 223.1.2.5, 67
dest: 255.255.255.255, 68
yiaddr: 223.1.2.4
transaction ID: 655
lifetime: 3600 secs

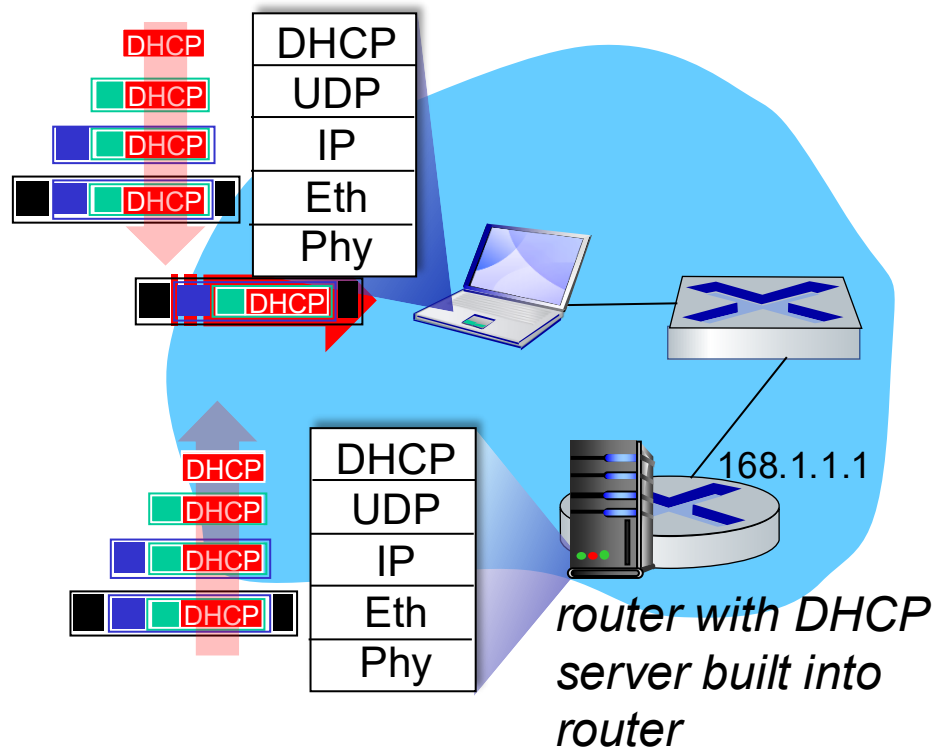
The two steps above can be skipped “if a client remembers and wishes to reuse a previously allocated network address” [RFC 2131]

DHCP: more than IP addresses

DHCP can return more than just allocated IP address on subnet:

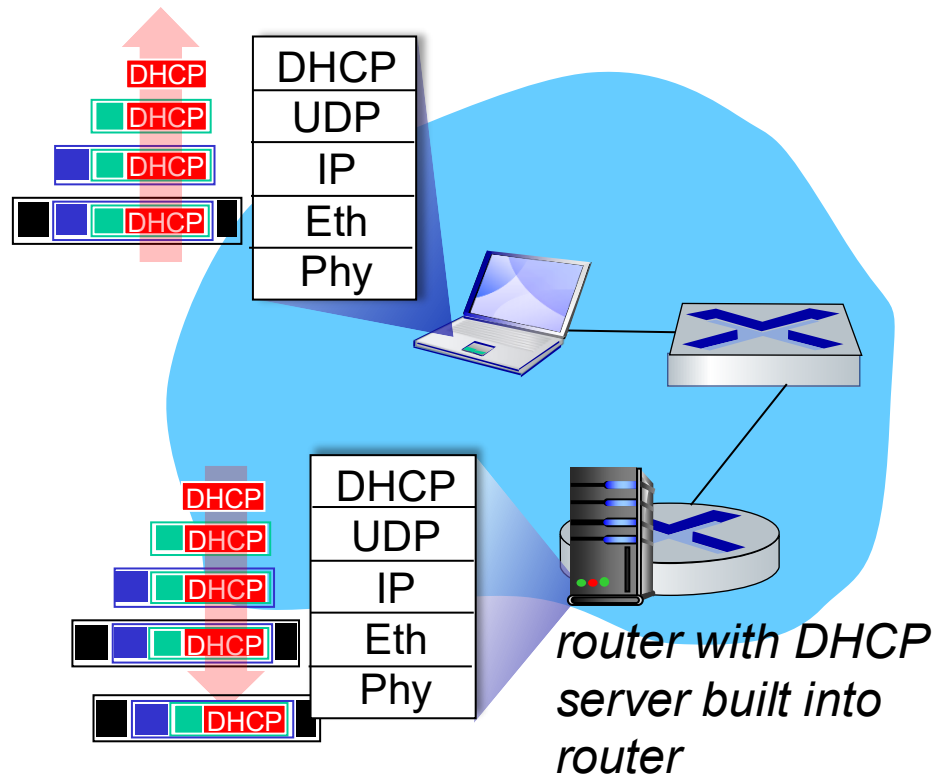
- address of first-hop router for client
- name and IP address of DNS sever
- network mask (indicating network versus host portion of address)

DHCP: example



- Connecting laptop will use DHCP to get IP address, address of first-hop router, address of DNS server.
- DHCP REQUEST message encapsulated in UDP, encapsulated in IP, encapsulated in Ethernet
- Ethernet frame broadcast (dest: FFFFFFFF) on LAN, received at router running DHCP server
- Ethernet de-mux'ed to IP de-mux'ed, UDP de-mux'ed to DHCP

DHCP: example



- DCP server formulates DHCP ACK containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- encapsulated DHCP server reply forwarded to client, de-muxing up to DHCP at client
- client now knows its IP address, name and IP address of DNS server, IP address of its first-hop router

IP addresses: how to get one?

Q: how does *network* get subnet part of IP address?

A: gets allocated portion of its provider ISP's address space

ISP's block 11001000 00010111 00010000 00000000 200.23.16.0/20

ISP can then allocate out its address space in 8 blocks:

Organization 0	<u>11001000</u>	<u>00010111</u>	<u>00010000</u>	00000000	200.23.16.0/23
Organization 1	<u>11001000</u>	<u>00010111</u>	<u>00010010</u>	00000000	200.23.18.0/23
Organization 2	<u>11001000</u>	<u>00010111</u>	<u>00010100</u>	00000000	200.23.20.0/23
...					
Organization 7	<u>11001000</u>	<u>00010111</u>	<u>00011110</u>	00000000	200.23.30.0/23

Network layer: “data plane” roadmap

- Network layer: overview
- IP: the Internet Protocol
 - Datagram format and IP Addressing
 - DHCP
 - ARP



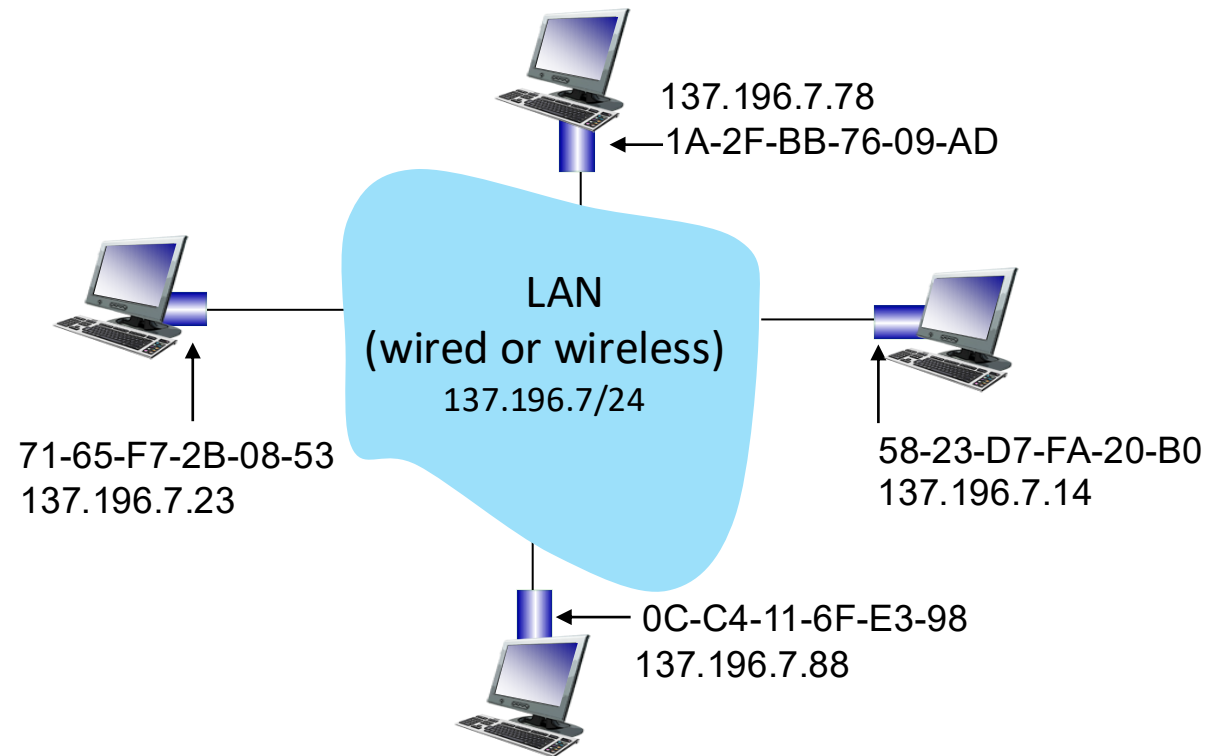
MAC (Media Access Control) addresses - Recap

- 32-bit IP address:
 - *network-layer* address for interface
 - used for layer 3 (network layer) forwarding
 - e.g.: 128.119.40.136
- MAC (or LAN or physical or Ethernet) address:
 - function: used “locally” to get frame from one interface to another physically-connected interface (same subnet, in IP-addressing sense)
 - 48-bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable
 - e.g.: 1A-2F-BB-76-09-AD
 - hexadecimal (base 16) notation
(each “numeral” represents 4 bits)

MAC addresses

each interface on LAN

- has unique 48-bit **MAC** address
- has a locally unique 32-bit IP address (as we've seen)

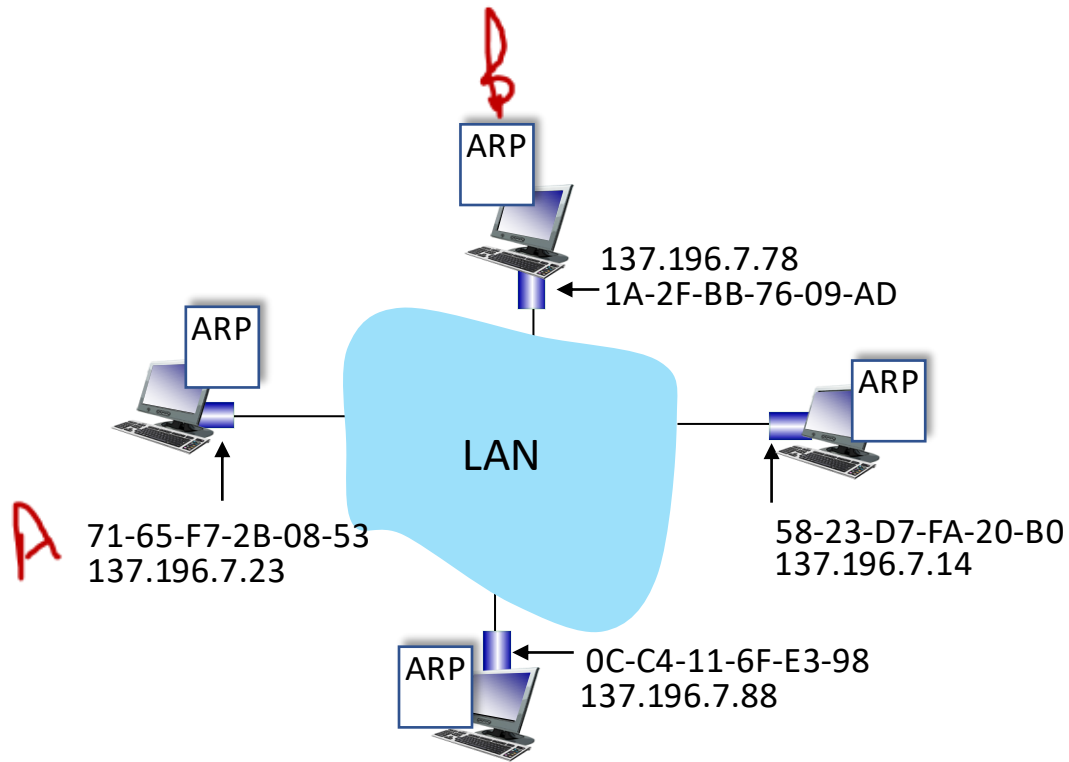


MAC addresses

- MAC address allocation administered by IEEE
- manufacturer buys portion of MAC address space (to assure uniqueness)
- analogy:
 - MAC address: like Social Security Number
 - IP address: like postal address
- MAC flat address: portability
 - can move interface from one LAN to another
 - recall IP address *not* portable: depends on IP subnet to which node is attached

ARP: address resolution protocol

Question: how to determine interface's MAC address, knowing its IP address?



ARP table: each IP node (host, router) on LAN has table

- IP/MAC address mappings for some LAN nodes:
< IP address; MAC address; TTL >
- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

ARP protocol in action

example: A wants to send datagram to B

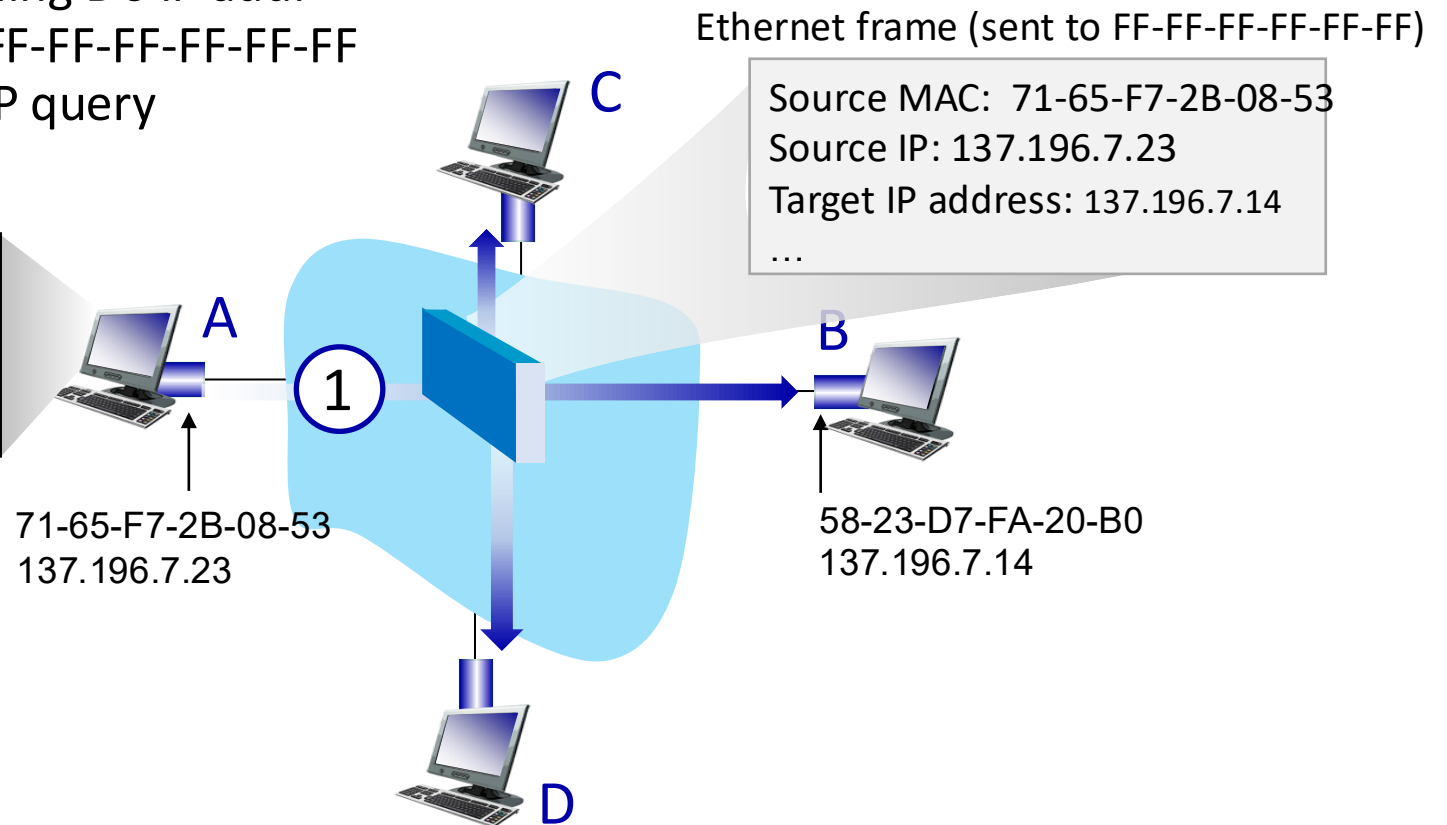
- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address

A broadcasts ARP query, containing B's IP addr

- ①
- destination MAC address = FF-FF-FF-FF-FF-FF
 - all nodes on LAN receive ARP query

ARP table in A

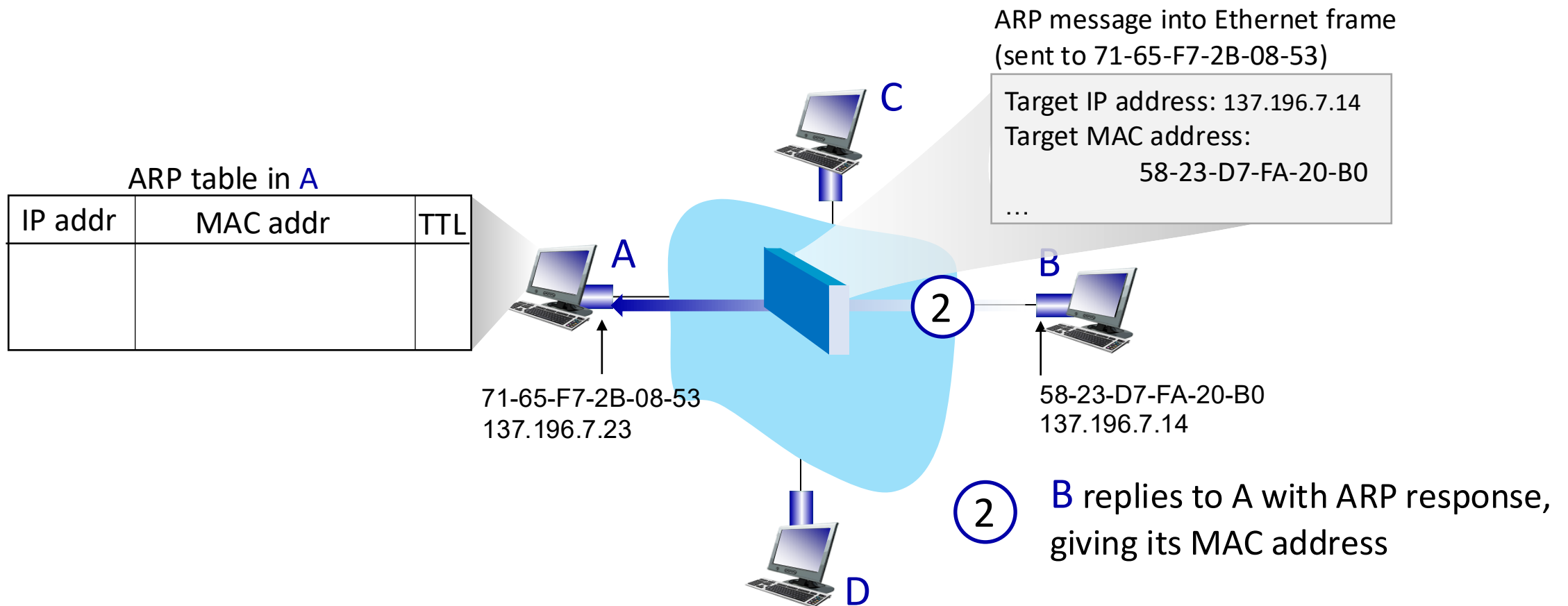
IP addr	MAC addr	TTL



ARP protocol in action

example: A wants to send datagram to B

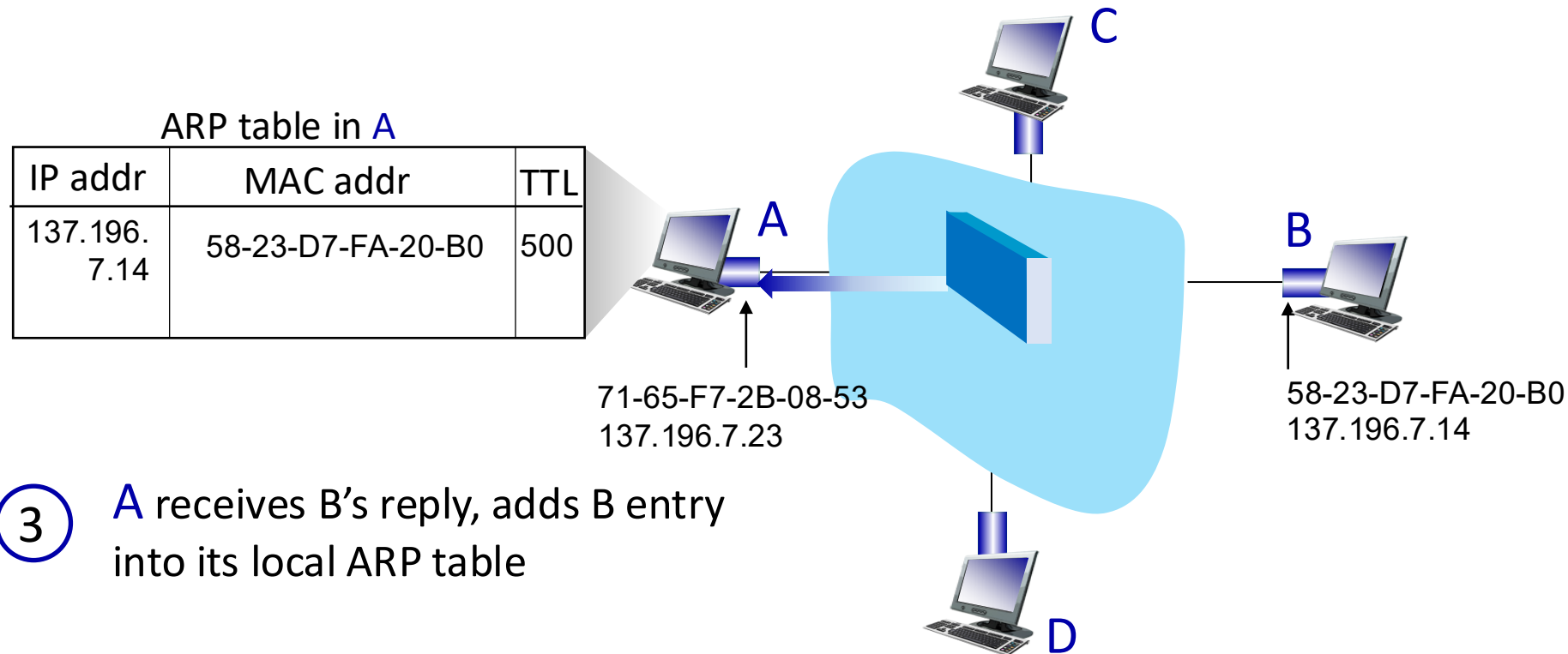
- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address



ARP protocol in action

example: A wants to send datagram to B

- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address



ARP Packet Format

Hardware Type		Protocol Type
Hardware length	Protocol length	Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)		
Target protocol address (For example, 4 bytes for IP)		

ARP Packet Format (28 Byte)

Field	Size (Bytes)	Description
Hardware Type	2	Specifies the link-layer protocol type. (1 = Ethernet)
Protocol Type	2	Specifies the higher-layer protocol (0x0800 = IPv4).
Hardware Length	1	Length of the MAC address (6 for Ethernet).
Protocol Length	1	Length of the IP address (4 for IPv4).
Operation (Opcode)	2	Defines request (1) or reply (2).
Sender Hardware Address	6	MAC address of the sender (who is making the request).
Sender Protocol Address	4	IP address of the sender.
Target Hardware Address	6	MAC address of the target. In a request, this is 00:00:00:00:00:00.
Target Protocol Address	4	IP address whose MAC address is being requested.

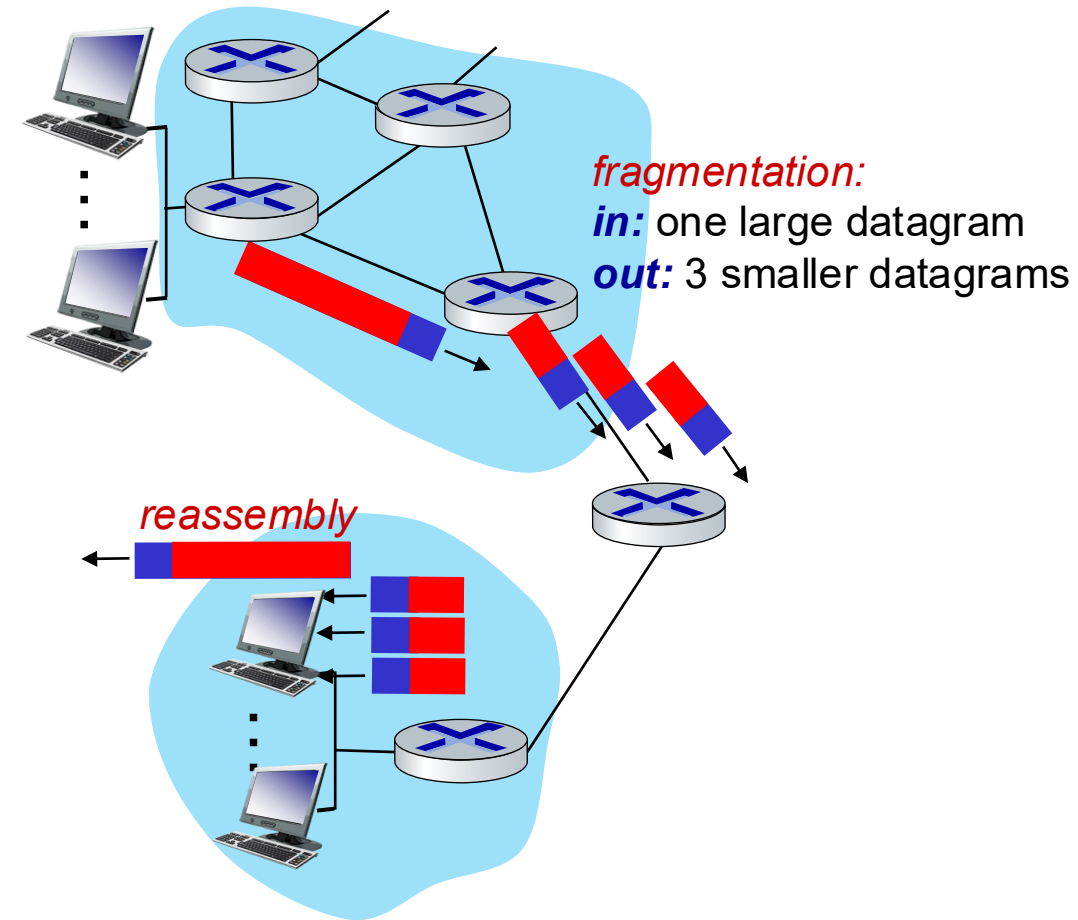
ARP Packet Format

```
▼ Frame 104: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface en0, id 0
  ► Interface id: 0 (en0)
    Encapsulation type: Ethernet (1)
    Arrival Time: Feb 26, 2020 18:43:06.784327000 EST
    [Time shift for this packet: 0.000000000 seconds]
    Epoch Time: 1582760586.784327000 seconds
    [Time delta from previous captured frame: 0.000209000 seconds]
    [Time delta from previous displayed frame: 0.000209000 seconds]
    [Time since reference or first frame: 2.048122000 seconds]
    Frame Number: 104
    Frame Length: 42 bytes (336 bits)
    Capture Length: 42 bytes (336 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:arp]
    [Coloring Rule Name: ARP]
    [Coloring Rule String: arp]
  ▼ Ethernet II, Src: XIAOMIEl_26:e5:0d (34:ce:00:26:e5:0d), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
    ► Destination: Broadcast (ff:ff:ff:ff:ff:ff)
    ► Source: XIAOMIEl_26:e5:0d (34:ce:00:26:e5:0d)
    Type: ARP (0x0806)
  ▼ Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: XIAOMIEl_26:e5:0d (34:ce:00:26:e5:0d)
    Sender IP address: 192.168.31.1
    Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Target IP address: 192.168.31.200
```

Additional Reading Slides

IP fragmentation/reassembly

- network links have MTU (max. transfer size) - largest possible link-level frame
 - different link types, different MTUs
- large IP datagram divided (“fragmented”) within net
 - one datagram becomes several datagrams
 - “reassembled” only at *destination*
 - IP header bits used to identify, order related fragments



IP fragmentation/reassembly

example:

- 4000 byte datagram
- MTU = 1500 bytes

	length =4000	ID =x	fragflag =0	offset =0	
--	-----------------	----------	----------------	--------------	--

*one large datagram becomes
several smaller datagrams*

1480 bytes in
data field

offset =
 $1480/8$

	length =1500	ID =x	fragflag =1	offset =0	
--	-----------------	----------	----------------	--------------	--

	length =1500	ID =x	fragflag =1	offset =185	
--	-----------------	----------	----------------	----------------	--

	length =1040	ID =x	fragflag =0	offset =370	
--	-----------------	----------	----------------	----------------	--

DHCP: Wireshark output (home LAN)

Message type: **Boot Request (1)**

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

request

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 0.0.0.0 (0.0.0.0)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 0.0.0.0 (0.0.0.0)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Server host name not given

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) **DHCP Message Type = DHCP Request**

Option: (61) Client identifier

Length: 7; Value: 010016D323688A;

Hardware type: Ethernet

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Option: (t=50,l=4) Requested IP Address = 192.168.1.101

Option: (t=12,l=5) Host Name = "nomad"

Option: (55) Parameter Request List

Length: 11; Value: 010F03062C2E2F1F21F92B

1 = Subnet Mask; 15 = Domain Name

3 = Router; 6 = Domain Name Server

44 = NetBIOS over TCP/IP Name Server

.....

Message type: **Boot Reply (2)**

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

reply

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 192.168.1.101 (192.168.1.101)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 192.168.1.1 (192.168.1.1)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Server host name not given

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) DHCP Message Type = DHCP ACK

Option: (t=54,l=4) Server Identifier = 192.168.1.1

Option: (t=1,l=4) Subnet Mask = 255.255.255.0

Option: (t=3,l=4) Router = 192.168.1.1

Option: (6) Domain Name Server

Length: 12; Value: 445747E2445749F244574092;

IP Address: 68.87.71.226;

IP Address: 68.87.73.242;

IP Address: 68.87.64.146

Option: (t=15,l=20) Domain Name = "hsd1.ma.comcast.net."